

CogPath: A Constraint-Guided Context-Reduction Framework for LLM-Based Test Generation

Anonymous Author(s)

Abstract

Automated unit test generation is essential for improving software reliability, yet existing techniques struggle to handle complex program semantics, especially when path reachability depends on implicit object states and inter-procedural interactions. Recent large language model (LLM)-based approaches can directly generate executable tests, but often fail to satisfy hidden preconditions that are not locally visible, resulting in low coverage. In this paper, we propose COGPATH, an LLM-driven framework that improves test generation by making implicit constraints explicit and reducing irrelevant context. Specifically, we introduce *Constraints-Hints* to expose path conditions as structured input requirements, and a coverage-driven backward slicing technique to retain only statements relevant to target branches. These components are integrated into an iterative pipeline that combines generation, execution, and repair. Experimental results on real-world Java projects show that COGPATH consistently achieves higher branch coverage, particularly for paths requiring precise object state construction. [xtz:need refine]

Keywords

Automated Unit Test Generation, Large Language Models, Software Testing, Program Analysis, Branch Coverage, Constraint-Guided Testing, Program Slicing

ACM Reference Format:

Anonymous Author(s). 2018. CogPath: A Constraint-Guided Context-Reduction Framework for LLM-Based Test Generation. *J. ACM* 37, 4, Article 111 (August 2018), 12 pages. <https://doi.org/XXXXXXX.XXXXXXX>

1 INTRODUCTION

Automated unit test generation plays a critical role in improving software reliability and reducing developer effort. Traditional techniques, such as symbolic execution and search-based software testing (SBST), have made substantial progress in systematically exploring input spaces and maximizing structural coverage [2, 5, 11, 12]. However, these approaches often face fundamental limitations in scalability and in handling complex program semantics. In particular, they struggle to capture implicit preconditions that arise from intricate object states and inter-procedural interactions, where the conditions required to reach a target path are established outside the local method scope through sequences of prior method calls [1].

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

© 2018 Copyright held by the owner/author(s). Publication rights licensed to ACM. ACM 1557-735X/2018/8-ART111 <https://doi.org/XXXXXXX.XXXXXXX>

Recent advances demonstrate that large language models (LLMs) can directly synthesize executable unit tests, significantly alleviating manual effort [? ?]. To handle complex methods and improve coverage, subsequent research mainly pursues two directions. The first is *feedback-driven refinement*: CoverUp [3] iteratively targets untested regions using dynamic coverage analysis, while YATE [?] repairs failing tests to enhance validity. The second direction emphasizes *structural decomposition and path sensitivity*. To simplify complex logic, HITS [18] prompts LLMs to decompose methods into fragments. Advancing this using program analysis, SymPrompt [18] extracts branch constraints from Control Flow Graphs (CFGs), whereas Panta [7] fuses static CFG analysis with dynamic coverage feedback to identify uncovered paths. Building upon this trajectory, JUnitGenie [9] achieves strict path sensitivity by isolating the minimal, path-specific context required for targeted generation.

Despite these promising results, a substantial coverage gap still remains, especially on complex branches whose reachability depends on non-local object states and long dependency chains. This suggests that the remaining challenge is not simply a lack of feedback or prompting sophistication, but a deeper mismatch between the information exposed to the LLM and the information actually required to cover a target branch. ① One important dimension of this mismatch is *what to satisfy*. In many cases, branch reachability depends on implicit preconditions scattered across long dependency chains, requiring the model to infer hidden constraints through multiple variable assignments and state transitions before it can synthesize a valid test. Existing approaches typically provide only the target branch structure and compress constraint inference and test synthesis into a single generation step, without any explicit intermediate output or independent verification of the inferred constraints. As a result, the LLM can easily skip or compress critical reasoning steps, producing tests that appear plausible but fail to satisfy the actual conditions required to reach the target branch. This problem becomes particularly severe when the underlying dependency chain is long or semantically complex. ② The second dimension concerns *what to attend to*. Existing methods usually expose the entire method body to the LLM, even though only a small subset of statements has real data or control dependence on the target branch. The resulting context is therefore structurally misaligned with the branch-coverage objective: coverage-critical dependencies are diluted by irrelevant code, attention is spread across non-essential statements, and the model must reason under a representation that over-emphasizes what is visible rather than what is truly necessary. This not only amplifies cognitive burden, but also makes precise state construction substantially less likely. Even without explicit distractors, LLM performance can already degrade as context length grows [4, 17]; in branch-oriented test generation, this effect is further magnified because the relevant context is often deeply buried inside much longer method bodies.

Taken together, these two issues suggest a systematic form of *cognitive misalignment* in LLM-based unit test generation: the information shown to the model is often misaligned with the information actually needed to cover a target branch. This misalignment has two complementary dimensions, namely *what to satisfy* and *what to attend to*.

To address this cognitive misalignment, we propose CogPATH, an LLM-driven framework for automated unit test generation that explicitly tackles both dimensions of the gap, by making critical constraints explicit and concentrating the model's attention on branch-relevant program context. First, **Constraint-Hints** addresses the *what to satisfy* dimension by making the execution conditions of uncovered paths explicit. Instead of forcing the LLM to deduce implicit constraints from raw code, this mechanism extracts path conditions and translates them into actionable guidance. It drastically reduces the model's reasoning difficulty by directly suggesting *how* to satisfy a target condition—such as invoking specific setter methods to establish the required object state. By surfacing these hidden cross-method preconditions, it actively guides the LLM to construct valid initialization sequences, ensuring complex dependencies are properly addressed rather than silently skipped. Second, *coverage-driven backward slicing* addresses the *what to attend to* dimension. Rather than supplying the full method body, the slicer isolates the minimal set of statements that genuinely influence an uncovered branch, including cross-method dependencies such as field initializations and setter methods. This restructures the prompt context around branch-relevant dependencies, preventing the model's attention from being diffused across irrelevant code. The slicing is triggered selectively when coverage plateaus, targeting branches that resist standard generation. Both components are integrated into an iterative loop of generation, execution, and repair. Coverage feedback drives target selection each round, progressively exposing branches where the current context remains misaligned with their reachability requirements, until coverage converges or the budget is exhausted.

Evaluation. We evaluate CogPATH on 14 Java projects from Defects4J v2.0.1, covering 130 classes and 2971 methods with cyclomatic complexity greater than 10. Baselines include SymPrompt, HITS, and Panta, representing complementary design philosophies. CogPATH achieves 44.41% average branch coverage, outperforming SymPrompt (21.14%), HITS (34.24%), and Panta (32.32%). Gains are most pronounced on logic-heavy subjects with rich cross-method dependencies: on Gson, CogPATH reaches 41.24% versus SymPrompt's 3.59%; on Jsoup, it improves over HITS by 36.4%.

Ablation study confirms the complementary roles of both components. Removing *Backward Slicing* drops coverage from 44.41% to 38.47%, with the largest degradation on context-heavy subjects (e.g., Csv: 52.29%→24.80%). Removing *Constraint-Hints* drops coverage to 39.79%, most severely on subjects with object-state dependencies (e.g., JacksonDatabind: 17.85%→9.33%). Finally, CogPATH generalizes across backbone LLMs, achieving 44.41% (QC3-30B), 83.47% (QW3.5-397B), and 80.12% (DS-V3.1), confirming that its benefit derives from restructuring branch-relevant information rather than relying on any specific model.

In summary, this paper makes the following contributions:

- We identify a key limitation of existing LLM-based test generation approaches in handling implicit, cross-method constraints.
- We propose *Constraints-Hints*, a mechanism that makes path conditions explicit and guides LLM reasoning.
- We introduce a coverage-driven backward slicing technique to eliminate irrelevant context and expose critical dependencies.
- We design an iterative framework that integrates generation, execution feedback, and repair for sustained coverage improvement.
- We empirically demonstrate the effectiveness of our approach on real-world software systems.

2 MOTIVATION

Recent LLM-based unit test generation approaches, such as SymPrompt [14], HITS [18], Panta [7], and JunitGenie [9], have achieved remarkable progress in automating test generation for complex methods, significantly reducing the manual effort required from developers. However, when the reachability of a target branch depends on *object states established by prior method calls*, existing approaches still struggle to guide LLMs in constructing the appropriate external context, even when augmented with CFG-based path analysis.

The fundamental limitation is that these approaches typically supply the *entire method body* as the LLM's input context. In real-world projects, however, the conditions necessary to reach a target branch are often not visible within the method body itself—they are implicit in a sequence of initialization and state-setting operations that must occur *before* the method is invoked [1]. The irrelevant statements in the method body not only consume the context window but also obscure these cross-method dependencies, leaving the LLM unable to infer the required preconditions.

A Motivating Example. [xtz:lack example fot backward slicing] Consider the `addValue()` method from Apache Commons Math [15], shown in Listing 1.

Listing 1: Target branch example from Apache Commons Math

```
// org.apache.commons.math3.stat.descriptive.  
    SummaryStatistics  
public void addValue(double value) {  
    sumImpl.increment(value);  
    ...  
  
    if (meanImpl != mean) { // target branch  
        meanImpl.increment(value);  
    }  
    ...  
    n++;  
}
```

Suppose a test generation tool identifies that the branch at line 9 (`meanImpl != mean`) has not been covered and attempts to generate a test to reach it. Existing approaches typically provide the entire method body alongside CFG path information to the LLM. However, the LLM faces a critical inferential barrier: the condition `meanImpl != mean` can only be satisfied if `setMeanImpl()` was called *prior* to `addValue()`—a dependency that is entirely absent

from the method body. Consequently, the LLM tends to generate a test such as:

Listing 2: Typical LLM-generated test (target branch unreachable)

```
// meanImpl == mean always holds; target branch is
// never triggered
SummaryStatistics stats = new SummaryStatistics();
stats.addValue(1.0);
```

This test compiles and executes without error, yet the target branch is never triggered. The correct test requires establishing the necessary precondition *before* invoking `addValue()`:

Listing 3: Correct test (target branch reachable)

```
// Establish precondition via setMeanImpl() before
// calling addValue()
SummaryStatistics stats = new SummaryStatistics();
stats.setMeanImpl(new CustomMeanImpl()); // makes
// meanImpl != mean
stats.addValue(1.0); // target
// branch now reachable
```

This example illustrates two core challenges that existing approaches fail to address: (1) the method body contains numerous statements irrelevant to the target branch, which dilute the LLM’s attention and impede precise reasoning; and (2) the true precondition for reaching the target branch is a cross-method dependency invisible within the method body itself.

todo: bs example

Our Approach. To address these challenges, we propose two complementary mechanisms that reduce LLM reasoning complexity from the perspectives of *information reduction* and *condition explicitness*:

- **Backward Slicing.** Starting from the target branch condition `meanImpl != mean`, we perform backward slicing to trace all statements with data or control dependencies on `meanImpl` and `mean`, including field declarations, default assignments in the constructor, and the definition of `setMeanImpl()`. This eliminates six irrelevant increment calls and other unrelated branches from the context, allowing the LLM to focus on statements that govern branch reachability.
- **Constraint Hints.** Building on the slice, we explicitly provide the LLM with the path condition and how it can be established, e.g., “To trigger the target branch, `meanImpl != mean` must hold; this can be achieved by calling `setMeanImpl(customImpl)` before `addValue()`.” Rather than requiring the LLM to infer implicit cross-method dependencies from source code, this mechanism directly guides the LLM toward constructing the correct initialization sequence.

Together, these two mechanisms systematically reduce the difficulty of reasoning about complex branch conditions, enabling LLMs to generate tests that reach previously uncovered paths.

3 Methodology

[xtz:need refine the logic and writing]

3.1 Overview

COGPATH is a coverage-guided iterative framework for automated unit test generation, illustrated in Figure 1. Given a method under test and an initial test suite, COGPATH builds the CFG, enumerates feasible paths, collects dynamic coverage feedback to identify uncovered branches, and then enters an iterative loop to systematically handle these branches. Each iteration proceeds in three steps. First, the framework selects an uncovered path and applies *Constraints-Hints* to extract explicit branch conditions as structured hints for the generation prompt. Second, if coverage has stagnated, *Backward Slicing* replaces the full method body with the minimal set of branch-relevant statements. Third, the LLM generates candidate tests, which are compiled and executed; passing tests are merged into the suite, and failing tests enter a repair phase. Coverage is then re-measured to guide the next iteration. The loop terminates when target coverage is reached, the iteration budget is exhausted, or coverage fails to improve for a configurable number of consecutive iterations.

In the following sections, we will provide detailed introductions about COGPATH.

~~COGPATH is a coverage-guided iterative framework for automated unit test generation. The overall workflow is illustrated in Figure 1. Starting from the method under test (MUT) and an initial test suite (possibly empty), the framework performs static analysis to construct the control-flow graph and enumerate feasible execution paths. Dynamic execution feedback is then collected to identify uncovered lines and branches.~~

~~To efficiently exercise uncovered paths, COGPATH integrates two complementary mechanisms. First, a *Constraints-Hints* module derives explicit branch conditions along a selected path and exposes them as structured hints in the test generation prompt, making hidden input requirements visible to the LLM. Second, a *Backward Slicing* module is triggered when coverage improvement stagnates, extracting the minimal set of statements that influence a stubborn uncovered path and supplying them as targeted context.~~

~~Each iteration proceeds as follows. The framework selects uncovered paths, optionally derives constraint hints, and invokes the test generation LLM to produce candidate tests. These candidates are compiled and executed to validate their correctness. Failing tests are optionally forwarded to a repair phase. Successfully validated tests are incorporated into the test suite and coverage is re-measured, closing the feedback loop. The process terminates when the target coverage is reached, the iteration budget is exhausted, or coverage ceases to improve for a configurable number of iterations.~~

3.2 Constraints-Hints for Path Solving

Constraints-Hints addresses the implicit precondition problem by decoupling constraint inference from test synthesis. Rather than asking the LLM to simultaneously reason about reachability and generate code, we introduce a dedicated constraint-solving step that makes branch preconditions explicit before generation begins. ~~For methods with complex branch logic, directly instructing an LLM to cover a specific execution path often fails because branch predicates are buried within nested conditionals, method calls, or arithmetic expressions. Consequently, the model cannot reliably infer the precise input conditions required to reach the desired~~

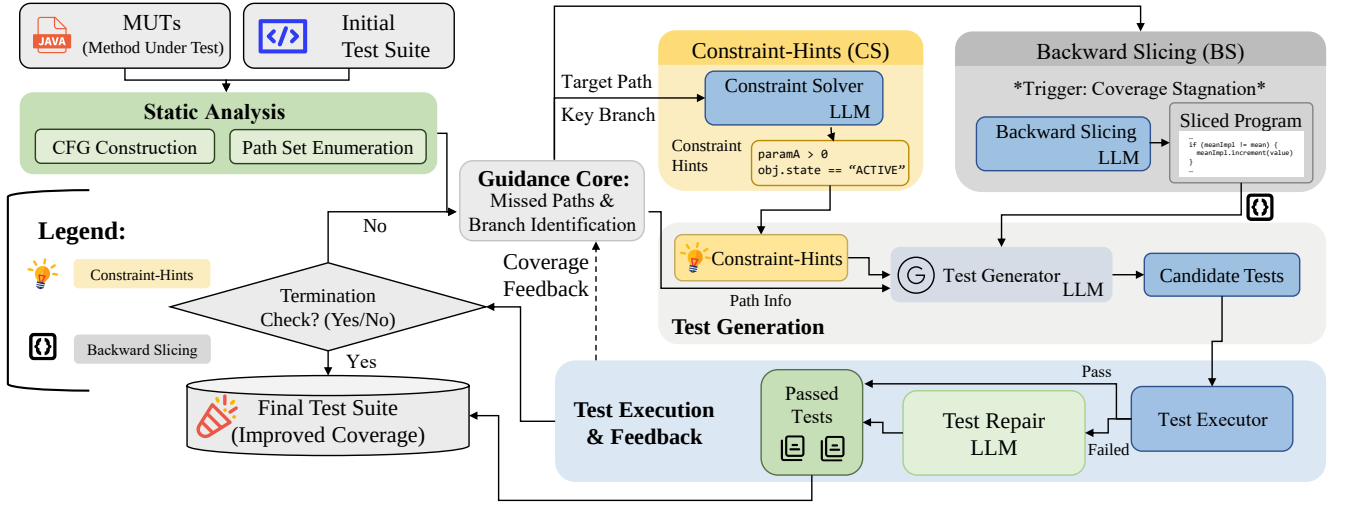


Figure 1: Overview of the coverage-guided test generation framework.

path. To address this limitation, we introduce *Constraints-Hints*, a mechanism that translates implicit branch conditions into explicit and structured input requirements.

Path formulation. Let $P = \langle b_1, b_2, \dots, b_k \rangle$ denote the sequence of branch decisions along a target execution path π , where each predicate $b_i(\mathbf{x}) \in \{\text{true}, \text{false}\}$ is evaluated over the program input vector $\mathbf{x}$. Covering the path requires an input assignment $\mathbf{x}^*$ that satisfies all branch predicates along the path:

$$\mathbf{x}^* \in \mathcal{F}(\pi) = \{\mathbf{x} \mid b_1(\mathbf{x}) \wedge b_2(\mathbf{x}) \wedge \dots \wedge b_k(\mathbf{x})\}. \quad (1)$$

Rather than solving this constraint system symbolically—which can be computationally expensive and brittle under complex object states or indirect method calls—COGPATH employs an LLM-based constraint solver to approximate the feasible input space as a lightweight set of interpretable conditions:

$$\mathcal{C}(\pi) = \{c_j \mid c_j \text{ is a necessary condition on } \mathbf{x}\}. \quad (2)$$

The first uncovered branch along the path, denoted as b^* , is identified from the coverage report and provided to the solver as additional context, guiding the model toward the most immediate obstacle preventing the path from being executed.

Integration. The constraint-solving prompt is generated from a configurable template that incorporates the source code, the branch sequence of π , and the uncovered branch b^* . The solver produces $\mathcal{C}(\pi)$ as a list of conditions such as `paramA > 0` or `obj.status == "ACTIVE"`. These conditions are appended to the test generation prompt, transforming an opaque path coverage objective into a concrete specification of required inputs.

Example. Consider the `addValue` method, which contains three conditional branches dispatching to overridable statistical implementations. An existing test suite covers only the false branches

of all three predicates, meaning `meanImpl`, `varianceImpl`, and `geMeanImpl` all refer to their default implementations. The first uncovered branch is `(meanImpl != mean)`. The constraint solver identifies that satisfying this predicate requires replacing the default implementation before invoking `addValue`. This object-state precondition is expressed as a constraint in $\mathcal{C}(\pi)$:

$$c : \text{meanImpl} \neq \text{mean}$$

which implies that a call such as `setMeanImpl(...)` must occur prior to invoking `addValue`. By making this requirement explicit, the test generation LLM can construct the correct setup sequence and cover the previously unreachable branch.

3.3 Backward Slicing for Coverage Plateaus

Despite coverage-guided iterative generation, certain branches may remain persistently uncovered, particularly in methods with high cyclomatic complexity or deeply nested control flow. In such cases, generative exploration often fails to infer the precise conditions required to execute the target path. To address this challenge, COGPATH incorporates a coverage-driven backward slicing mechanism that is activated only when coverage improvement stalls. Coverage improvement does not always progress smoothly: certain branches remain persistently uncovered even after multiple iterations, typically because their reachability depends on subtle state dependencies that standard generation fails to capture. When such a plateau is detected, COGPATH triggers a backward slicing step targeted at these stubborn branches, replacing the full method body with the minimal context actually relevant to the uncovered path.

Relationship to prior work. Program slicing has long been used to identify the subset of program statements relevant to a particular computation [19]. Recent work such as NS-Slicer [14] demonstrates that pre-trained language models can approximate static slices by learning variable-statement dependencies. In the context of LLM-based test generation, HTS [18] applies slicing to guide test synthesis;

however, its decomposition is driven by structural heuristics rather than coverage feedback, meaning the resulting segments are not targeted toward any specific uncovered branch. Furthermore, for methods with high cyclomatic complexity, the number of generated segments can grow substantially, incurring significant token overhead regardless of whether the corresponding code has already been covered. In contrast, COGPATH performs *path-aware backward slicing* triggered only when coverage improvement stalls, ensuring that each slice is both necessary and focused on the precise dependencies of the uncovered branch.

Slice extraction. Given an uncovered path π , the framework extracts the subset of statements that may influence the execution of that path:

$$\text{Slice}(P, \pi) = \{s_i \in P \mid s_i \rightsquigarrow \pi\}. \quad (3)$$

A dedicated LLM is then invoked with the uncovered code segment, the focal method source code, and dependency context from related classes. The model returns a structured response containing the relevant slice statements, required input values, necessary object states, and potential setup operations.

Test generation from slices. The extracted slice information is supplied as additional context to the test generation prompt. The LLM then generates targeted unit tests that incorporate the identified prerequisites directly. These tests are compiled, executed, and measured using the same validation pipeline as regular test candidates, allowing the framework to recover from coverage plateaus while maintaining the overall iterative workflow.

3.4 [xtz:Iterative Workflow of COGPATH] LLM-based Test Generation and Repair

The complete workflow of COGPATH is formalized in Algorithm 1. The framework accepts the source file and an initial test suite as input, and iteratively generates, validates, and repairs tests until a coverage target is met or a termination condition is triggered.

Initialization. The framework extracts test-scope dependencies, constructs the CFG and method dictionary from the source file, and measures baseline coverage on the initial suite (lines 1–4).

Path selection with Constraints-Hints. At each non-initial iteration, SELECTUNCOVEREDPATHS identifies candidate paths from the CFG that intersect with missed lines or branches, weighted by both coverage impact and visit history *pathHistory* (line 8). For each selected path π , the constraint solver derives $\mathcal{C}(\pi)$ by analyzing the branch sequence up to the first uncovered predicate b^* , and these conditions are embedded into the generation prompt (lines 9–13).

Backward slicing. When $\text{noIncr} \geq \beta \cdot \Lambda$, indicating a coverage plateau, the backward slicer is invoked on the remaining missed code segments. It returns structured prerequisite information—required inputs, object states, and method mocks—which is used to construct an additional batch of targeted tests $\mathcal{T}_s$ (lines 16–19).

Validation and repair. Each generated test is individually inserted into the suite and executed. Failing tests are collected with their error messages and forwarded to the repair phase, which re-invokes

Algorithm 1: Iterative Test Generation with Constraints-Hints and Backward Slicing

Input: *src*: source file, *suite*: initial test suite (possibly empty)
Output: Augmented test suite with improved coverage

```

deps ← EXTRACTDEPS();
cfg, methodDict ← BUILDCFG(src);
noIncr ← 0, iter ← 0;
cov, missed ← RUNCOVERAGE(suite);
while iter < maxIter and noIncr <  $\Lambda$  and cov <  $\tau$  do
    // Path selection and prompt construction
    if cov = 0 or iter = 0 then
        prompt ← BUILDBASELINEPROMPT(src, suite, deps);
        ; // baseline prompt
    else
         $\Pi$  ← SELECTPATHS(cfg, methodDict, missed);
         $\mathcal{C}$  ←  $\emptyset$ ;
        foreach path  $\pi \in \Pi$  do
             $b^*$  ← FIRSTUNCOVEREDBRANCH( $\pi$ , missed);
             $\mathcal{C}(\pi)$  ← CONSTRAINTSOLVER(src,  $\pi$ ,  $b^*$ );
            ; // Constraints-Hints
        end
        prompt ← BUILDPROMPT(src, suite, deps,  $\Pi$ ,  $\mathcal{C}$ );
    end
    // Test generation
     $\mathcal{T}$  ← GENERATETESTS(prompt);
    // Backward slicing for stubborn uncovered paths
    if noIncr  $\geq \beta \cdot \Lambda$  then
        slices ← BACKWARDSLICER(src, missed);
         $\mathcal{T}_s$  ←  $\mathcal{T} \cup \text{GENERATEFROMSLICES}(\text{slices}, \text{suite}, \text{deps})$ ;
    end
    // Validation and repair
    failed ← {};
    foreach t  $\in \mathcal{T}$  do
        if VALIDATEST(t, suite) then suite ← suite  $\cup \{t\}$ ;
        else failed ← failed  $\cup \{t\}$ ;
    end
    if failed  $\neq \emptyset$  then
        suite ← suite  $\cup \text{REPAIRANDVALIDATE}(\text{failed}, \text{src}, \text{suite}, \text{deps})$ ;
    end
    // Coverage update and termination check
    (cov', missed') ← RUNCOVERAGE(suite);
    if cov' > cov then noIncr ← 0;
    else noIncr ← noIncr + 1;
    cov ← cov', missed ← missed', iter ← iter + 1;
end
return suite;

```

the LLM with failure feedback to produce corrected versions. Successfully validated repaired tests are also incorporated into the suite (lines 21–32).

Termination. The loop exits when the coverage target τ is reached, the maximum iteration count *maxIter* is exhausted, or coverage fails to improve for Λ consecutive iterations.

4 EXPERIMENTS

4.1 Experimental Setup

In this section, we first introduce the construction and statistics of the evaluation dataset.

Dataset. To evaluate the effectiveness of COGPATH in generating unit tests for complex methods, we utilize real-world subjects from the Defects4J (v2.0.1) benchmark [8]. We selected the latest fixed version of each subject program to ensure compatibility with modern build environments. Following prior work [7], we excluded projects that do not use Maven or rely on deprecated dependencies (i.e., Chart, Mockito, and Closure). The remaining 14 subjects are compatible with Java 8+, Maven (v3.6.3), and JUnit (v4.13.2). To specifically assess performance on logic-heavy code, we focus on complex classes rather than entire projects. We identified non-abstract, public classes containing at least one method with Cyclomatic Complexity (CC) > 10 [10]. Furthermore, we manually excluded outliers such as classes with extreme complexity ($CC > 40$) caused by massive switch-statement patterns and those with minimal explicit control flow due to deep inheritance. [xtz:why? Does the baseline do the same thing?] The statistics of our evaluation dataset are summarized in Table 1.

Evaluation Metrics. To evaluate the effectiveness of the generated unit tests, we primarily focus on code coverage, which is the standard metric in automated test generation research [5, 12]. We report two specific metrics:

- **Line Coverage (LC):** The ratio of executable lines in the target class that are exercised by the generated test suite. It provides a baseline measure of how much of the implementation has been reached.
- **Branch Coverage (BC):** The ratio of executed decision outcomes (e.g., the true and false paths of an *if* statement) to the total number of possible branches. Given our focus on methods with high Cyclomatic Complexity ($CC > 10$), branch coverage is our primary metric, as it more accurately reflects the tool's ability to navigate complex logical structures and boundary conditions.

All coverage data in our experiments are collected using the JaCoCo library, ensuring consistent and reproducible measurements across all subjects and baselines.

Research Questions. To evaluate the effectiveness and design of COGPATH, we address the following three research questions:

- **RQ1: Effectiveness Comparison.** How does COGPATH perform compared to state-of-the-art LLM-based test generation approaches in terms of line and branch coverage on complex, logic-heavy methods?
- **RQ2: Contribution of Components.** What are the individual contributions of the Backward Slicing (BS) and Constraint Solver (CS) components to the overall performance of COGPATH?
- **RQ3: Model Generalizability.** Is COGPATH's effectiveness robust across different underlying Large Language Models, and how do their performances vary?

Baselines. To evaluate the performance of COGPATH, we compare it against three state-of-the-art (SOTA) LLM-based unit test

generation frameworks. These baselines represent diverse paradigms in the field: [xtz:only explain the method of baseline.]

- **SymPrompt [14]:** A prominent prompt-engineering approach that integrates symbolic execution concepts with LLMs. It utilizes path-oriented prompts to guide models in generating test inputs. We include SymPrompt to assess the effectiveness of COGPATH against methods relying solely on structured prompting without external constraint solving.
- **HITS [18]:** A high-performance framework emphasizing self-healing and iterative refinement. As a strong baseline for generating compilable and high-coverage tests, HITS allows us to evaluate COGPATH's capability in navigating complex logical branches where iterative feedback is critical.
- **Panta [7]:** A SOTA tool that leverages advanced path analysis and context-aware prompting. Panta excels at capturing data dependencies within the Class Under Test (CUT). Comparing against Panta demonstrates the advantages of COGPATH's synergy between branch selection and logical constraint satisfaction.

Due to the absence of several critical components in the official HITS repository, we faithfully re-implemented its workflow by strictly following the prompt templates defined in the original paper and adhering to the procedural logic of the available open-source fragments. To validate the fidelity of our re-implementation, we ran it using GPT-3.5-turbo—the backbone model adopted in the original HITS paper—on the Cli project; the resulting branch coverage deviates by less than 2% from the figures reported in the original paper, confirming the reliability of our re-implementation.

For a fair comparison, all baselines were configured using the same backbone model as COGPATH in each experiment. Specifically, RQ1 adopts Qwen3-Coder-30B (hereafter QC3-30B) as the backbone, while RQ2 adopts Qwen3-Coder-30B (hereafter QC3-30B). The use of different models across RQs reflects the distinct focuses of each evaluation: RQ1 prioritizes generation quality with a larger model, whereas RQ2 isolates the contribution of individual components under a more controlled, cost-efficient setting. Specifically, both RQ1 and RQ2 adopt Qwen3-Coder-30B (hereafter QC3-30B) as the unified backbone model, enabling a controlled and cost-efficient comparison across all experimental conditions. Furthermore, all methods were provided with identical CUT information and dependency contexts to ensure that performance differences are attributable to the frameworks themselves rather than external information.

Contextual Information. [xtz:why?] To evaluate the inherent logic reasoning and code understanding capabilities of the models, we adopt a restricted context strategy for all test generation tasks. Specifically, for each method under test (MUT), the prompt only includes the source code of its declaring class and the names of the associated import packages. No additional project-level information, such as dependency class implementations or global configuration files, is provided. This setup ensures that the performance of COGPATH and the baselines is assessed under a consistent and challenging information-constrained environment, reflecting a realistic scenario where cross-file dependencies may be complex or partially inaccessible.

Table 1: Dataset Metrics with #MUTs

Identifier	Project	Version	#C	#MUTs	CC_{max}	CC_{min}	CC_{med}
Cli	Cli-40f	1.5-SNAP	2	31	11	11	11
Codec	Codec-18f	1.11-SNAP	7	78	31	11	13
Collections	Collections-28f	4.2-SNAP	5	218	32	11	16
Compress	Compress-47f	1.16-SNAP	9	65	22	11	13
Csv	Csv-16f	1.6-SNAP	3	72	25	11	14
Gson	Gson-16f	2.8.2-SNAP	4	75	37	11	22
JCore	JacksonCore-26f	2.10.0-SNAP	9	161	30	11	14
JDatabind	JacksonDatabind-112f	2.9.9	9	371	22	11	15
JXml	JacksonXml-5f	2.9.6-SNAP	4	118	29	12	22
Jsoup	Jsoup-93f	1.12.2-SNAP	8	171	22	11	12
JxPath	JxPath-22f	1.4-SNAP	12	115	32	11	13
Lang	Lang-4f	3.2-SNAP	17	586	30	11	16
Math	Math-2f	3.3-SNAP	30	452	31	11	14
Time	Time-13f	2.1	11	458	30	11	17
Total/Avg			130	2971	27.4	11.1	15.1

Configurations. Our experiments employ two backbone LLMs depending on the research question. For RQ1 (effectiveness) and RQ2 (ablation study), we adopt Qwen3-Coder-30B (QC3-30B), deployed locally on a server equipped with an NVIDIA H800 GPU (80 GB); this lighter model is chosen to enable a controlled, cost-efficient comparison. To assess model generalizability in RQ3, we evaluate four representative LLMs: QC3-30B, Qwen3.5-397B-A17B (QW3.5-397B), GPT-5.4-Mini, and DeepSeek-V3.1 (DS-V3.1), ranging from medium-scale local deployment to large-scale models accessed via their respective official APIs. To ensure reproducibility and minimize stochastic variance, all models are configured with a temperature of 0.2 and top- p of 1.0. The key hyperparameters of CogPATH are configured as follows: the maximum number of iterations $maxIter$ is set to $[\]$, the no-improvement tolerance Λ is set to $[\]$, the backward slicing trigger threshold β is set to $[\]$, and the target coverage τ is set to $[\]\%$. All experiments start from an empty initial test suite.

4.2 RQ1: Effectiveness of Our Approach

To answer RQ1, we compare CogPATH against three state-of-the-art LLM-based test generation frameworks—SymPrompt, HITS, and Panta—on 14 real-world Java projects from Defects4J (v2.0.1). The evaluation focuses on both line coverage (LC) and branch coverage (BC) of complex methods, with Cyclomatic Complexity $CC > 10$. To answer RQ1, we compare CogPATH against three state-of-the-art baselines on the selected dataset. Results are summarized in Table 2.

4.2.1 Overall Performance. As shown in Table 2, CogPATH achieves the highest average LC and BC across all projects. Specifically, it attains an average BC of 44.41%, outperforming SymPrompt (21.14%), HITS (34.24%), and Panta (32.32%). This demonstrates that the combination of constraint hints and backward slicing allows CogPATH to more effectively navigate complex control flows and satisfy cross-method preconditions, compared with baselines that rely solely on prompt engineering or CFG-based analysis. The improvement over SymPrompt

is particularly substantial (+89.3%), reflecting the limitation of approaches that lack explicit program analysis. Gains over HITS and Panta are smaller yet consistent (+16.9% and +23.8%, respectively), confirming that CogPATH’s enhancements provide complementary benefits beyond existing path enumeration or context-aware prompting techniques. Gains over the two strongest baselines—HITS (+10.2%) and Panta (+12.1%)—confirm that jointly reducing irrelevant context and making implicit preconditions explicit addresses coverage gaps that iterative refinement and context-aware prompting alone cannot close. The margin widens substantially against SymPrompt (+23.3%), underscoring the necessity of explicit program analysis when navigating logic-heavy control flows.

4.2.2 Performance on High-Complexity Projects. The advantages of CogPATH are most pronounced on projects with high cyclo-matic complexity and deeper inter-method dependencies. For example:

- Gson ($CC_{avg} = 23.25$, highest in the benchmark): BC of 41.24%, while SymPrompt reaches only 3.59%, highlighting the difficulty of context-unaware methods in handling complex branching logic.
- JCore ($CC_{avg} = 16.33$): BC improves from 18.51% (HITS) to 35.54% (CogPATH), a gain of 17%, demonstrating that backward slicing effectively isolates branch-relevant statements.
- Jsoup ($CC_{avg} = 14.13$): BC rises from 17.36% (HITS) to 52.55% (CogPATH), a 36.4% improvement, underscoring the impact of constraint hints in constructing necessary object states to reach target branches.

For instance, on Gson ($CC_{avg} = 23.25$, the highest in the benchmark), CogPATH achieves a BC of 41.24% while SymPrompt reaches only 3.59%; on JCore ($CC_{avg} = 16.33$), BC rises from 18.51% (HITS) to 35.54%; and on Jsoup ($CC_{avg} = 14.13$), BC improves from 17.36% (HITS) to 52.55%. These gains collectively demonstrate that reducing irrelevant context via backward slicing and surfacing implicit preconditions via constraint hints are both essential for navigating the dense branching logic and cross-method dependencies that characterize high-complexity projects.

Table 2: Coverage Across Approaches (Updated CogPATH)

Project	#C	CC_{avg}	SymPrompt		HITS		Panta		CogPATH	
			LC(%)	BC(%)	LC(%)	BC(%)	LC(%)	BC(%)	LC(%)	BC(%)
Cli	2	11.00	17.93	9.02	73.50	59.58	63.11	37.98	84.32	70.73
Codec	7	16.14	29.68	24.96	65.73	60.25	60.53	54.82	76.97	69.60
Collections	5	18.60	32.63	32.42	68.23	70.00	33.44	32.40	40.46	34.63
Compress	9	15.33	11.84	11.34	31.64	25.24	34.36	28.45	36.09	29.05
Csv	3	16.67	37.69	23.79	28.95	14.73	50.98	37.99	62.27	52.29
Gson	4	23.25	13.16	3.59	31.54	21.25	21.81	15.20	63.82	41.24
JCore	9	16.33	23.62	21.44	19.85	18.51	26.24	21.97	41.72	35.54
JDatabind	9	15.33	9.99	6.06	18.33	14.44	18.17	14.34	23.35	17.85
JXml	4	21.50	17.45	18.56	18.59	18.84	31.89	23.55	20.61	17.08
Jsoup	8	14.13	24.72	19.13	27.82	17.36	49.80	39.25	66.43	52.55
JXPath	12	15.50	12.01	9.48	23.11	22.76	34.10	29.35	42.28	35.84
Lang	17	16.88	43.98	40.42	62.82	59.77	39.26	36.61	59.21	52.14
Math	30	16.47	38.55	33.70	30.87	26.38	46.40	39.16	59.60	54.02
Time	11	17.36	53.37	42.06	63.32	50.23	46.74	41.42	66.79	59.25
Average			26.19	21.14	40.31	34.24	39.77	32.32	53.14	44.41

These results indicate that CogPATH excels particularly in logic-heavy methods where complex object interactions and conditional dependencies are prevalent.

4.2.3 Cases of Limited Improvement. [xtz:move to discussion] While CogPATH achieves the best average performance, there are a few projects where it does not outperform all baselines. For instance, Collections shows HITS achieving higher BC (70.00%) compared to CogPATH (34.63%), and Compress has similar trends. Manual inspection suggests that these projects contain a high proportion of branches governed by simple value-range conditions rather than complex cross-method states. In such cases, backward slicing occasionally removes auxiliary context that would otherwise help the LLM, slightly reducing coverage. This points to a potential area for refinement in slice granularity selection.

4.2.4 Summary. In summary, CogPATH demonstrates robust effectiveness across 14 real-world projects, achieving significant gains over state-of-the-art baselines, particularly in high-complexity and logic-intensive methods. The combined use of backward slicing and constraint hints enables the LLM to reach branches that are otherwise inaccessible to prompt-only or context-limited approaches.

Answer to RQ1

CogPATH outperforms all three baselines on average, achieving a BC of 44.41%, representing improvements of 10.2%–23.3% over baselines. Its advantage is most pronounced in projects with high cyclomatic complexity and cross-method dependencies (e.g., +20% on Gson, +14% on JCore). In a minority of projects where branches depend primarily on simple value conditions, existing tools remain competitive, highlighting opportunities for further refinement in slice granularity.

4.3 RQ2: Ablation Study

4.4 Contribution of Different Components

To evaluate the individual contributions of Backward Slicing (BS) and Constraint Hints (CS), we conducted an ablation study with three degraded variants: **w/o BS** (removing backward slicing), **w/o CS** (removing constraint hints), and **w/o CS+BS** (removing both). All variants share the same iterative generation loop and are evaluated under an identical backbone (QC3-30B). Results are summarized in Table 3.

4.4.1 Overall Ablation Results. Both components contribute independently. Removing BS alone causes an average BC drop of **5.94 percentage points** (44.41% → 38.47%), while removing CS alone results in a drop of **4.62 pp** (44.41% → 39.79%). Removing both components yields the largest degradation of **7.84 pp** (44.41% → 36.57%), confirming that BS and CS address complementary aspects of the problem: BS eliminates irrelevant context that misleads the LLM, while CS makes cross-method preconditions explicit so that the LLM can construct the required object state. Notably, the combined degradation (7.84 pp) is smaller than the sum of individual degradations (10.56 pp), suggesting that the two components partially compensate for each other when one is absent.

BS is particularly critical for context-heavy methods. The impact of BS is most pronounced in projects where method bodies contain many statements unrelated to the target branch. On Csv, removing BS causes BC to drop from 52.29% to 24.80% (−27.49 pp), the largest per-project degradation in the study. This aligns with our motivation: when irrelevant statements dominate the focal method, providing the full unsliced context actively distracts the LLM from the branch-relevant logic.

CS is critical when preconditions span method boundaries. CS contributes most on projects where the target branch condition depends on prior method calls. On JacksonDatabind, removing CS reduces BC from 17.85% to 9.33% (−8.52 pp, a relative decline of 47.7%), indicating that without explicit constraint hints the LLM

Table 3: Ablation Study of CogPATH Components

Project	CC_{avg}	CogPath		w/o CS		w/o BS		w/o CS+BS	
		LC(%)	BC(%)	LC(%)	BC(%)	LC(%)	BC(%)	LC(%)	BC(%)
Cli	11.00	84.32	70.73	84.74	68.75	68.69	57.68	72.62	56.54
Codec	16.14	76.97	69.60	54.86	50.68	66.77	57.94	66.34	56.53
Collections	18.60	40.46	34.63	39.51	38.59	32.42	36.60	37.43	34.45
Compress	15.33	36.09	29.05	31.22	28.58	25.43	19.04	23.45	18.11
Csv	16.67	62.27	52.29	46.18	36.38	36.47	24.80	60.54	47.07
Gson	23.25	63.82	41.24	43.94	27.73	49.40	31.85	40.92	26.52
JacksonCore	16.33	41.72	35.54	30.58	27.31	45.39	39.20	22.97	21.07
JacksonDatabind	15.33	23.35	17.85	12.44	9.33	33.74	26.55	19.59	15.54
JacksonXml	21.50	20.61	17.08	23.80	18.61	35.30	26.84	26.79	23.08
Jsoup	14.13	66.43	52.55	61.48	46.64	69.11	53.76	59.00	47.11
JXPath	15.50	42.28	35.84	47.56	41.96	28.06	22.35	34.94	31.44
Lang	16.88	59.21	52.14	66.05	63.04	56.84	51.38	57.12	53.91
Math	16.47	59.60	54.02	63.42	56.61	38.60	32.66	43.23	39.90
Time	17.36	66.79	59.25	46.29	42.81	64.99	57.89	43.21	40.65
Average		53.14	44.41	46.58	39.79	46.51	38.47	43.44	36.57

is unable to construct the object state necessary to reach the focal branch.

Anomalies and discussion. In two projects (*JacksonCore* and *Jsoup*), the w/o-BS variant achieves marginally higher coverage than the full CogPATH. We attribute this to cases where backward slicing inadvertently removes type information required for mock construction, causing the LLM to fall back on simpler yet occasionally effective test patterns. This sensitivity to slice granularity is a known limitation and remains a direction for future refinement.

4.4.2 Iterative Coverage Improvement. [xtz:add iteration curve of llm+constraint hints]

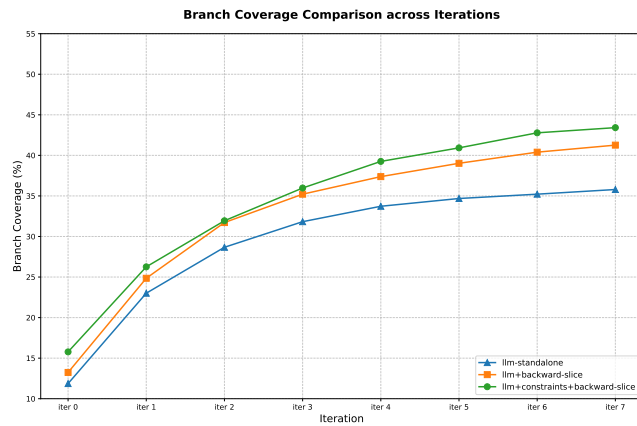


Figure 2: Branch coverage improvement over iterations for CogPATH and its three ablated variants.

Figure 2 traces branch coverage across iterations for all four configurations. CogPATH consistently achieves the highest coverage at every iteration and exhibits the steepest growth curve, demonstrating that both BS and CS contribute not only to final coverage

but also to the efficiency of iterative improvement. In contrast, removing either component leads to slower convergence, and removing both produces the flattest growth trajectory, underscoring the necessity of each component for sustained progress.

Answer to RQ2

Both BS and CS make independent and complementary contributions to CogPATH. BS provides the larger share of improvement by eliminating irrelevant context (−5.94 pp when removed), while CS explicitly guides the LLM toward satisfying cross-method preconditions (−4.62 pp when removed). Removing both components degrades BC by 7.84 pp on average, with per-project drops reaching up to 27.49 pp (Csv), confirming that neither component is redundant. The iterative growth curves further show that both components contribute to the efficiency of coverage accumulation across generation rounds.

4.5 RQ3: Performance Comparison of Different LLMs

In this research question, we evaluate how CogPATH generalizes across different large language models (LLMs), and investigate the impact of model capacity on test generation effectiveness. Specifically, we conduct experiments on four representative LLMs, ranging from medium-scale (QC3-30B) to large-scale models (QW3.5-397B and DS-V3.1).

Table 4 reports the line coverage (LC) and branch coverage (BC) achieved by each model across 14 real-world Java projects. Overall, CogPATH consistently achieves strong performance across all models, demonstrating its robustness to different underlying LLM architectures.

However, model capacity has a significant impact on coverage effectiveness. Larger models, such as QW3.5-397B and DS-V3.1, achieve substantially higher average branch coverage (83.47% and

Table 4: Model Generalization Across Different LLMs

Project	#C	CC	QC3-30B		GPT-5.4-Mini		QW3.5-397B		DS-V3.1	
			LC(%)	BC(%)	LC(%)	BC(%)	LC(%)	BC(%)	LC(%)	BC(%)
Cli	2	11.00	84.32	70.73	97.86	86.18	98.82	92.63	98.82	91.93
Codec	7	16.14	76.97	69.60	89.78	82.61	98.32	95.30	93.60	88.46
Collections	5	18.60	40.46	34.63	98.31	94.14	98.16	94.43	98.02	94.20
Compress	9	15.33	36.09	29.05	87.15	80.29	86.92	80.90	81.32	76.04
Csv	3	16.67	62.27	52.29	87.42	77.76	95.71	87.42	85.70	77.53
Gson	4	23.25	63.82	41.24	90.26	83.06	92.88	87.97	92.13	82.86
JCore	9	16.33	41.72	35.54	63.71	57.83	83.25	76.77	66.62	57.76
JDatabind	9	15.33	23.35	17.85	76.16	68.78	74.93	69.81	84.06	77.44
JXml	4	21.50	20.61	17.08	73.41	74.20	62.20	63.65	66.06	65.22
Jsoup	8	14.13	66.43	52.55	93.37	82.96	95.87	87.49	87.15	79.98
JXPath	12	15.50	42.28	35.84	81.72	75.11	71.30	66.22	76.89	73.21
Lang	17	16.88	59.21	52.14	90.65	84.32	91.72	87.44	88.21	85.02
Math	30	16.47	59.60	54.02	88.89	81.82	92.96	87.47	91.69	85.41
Time	11	17.36	66.79	59.25	94.18	87.63	96.94	91.08	94.71	89.92
Average			53.14	44.41	86.63	79.76	88.57	83.47	86.07	80.36

80.36%, respectively), compared to 44.41% for QC3-30B. This gap highlights the importance of advanced reasoning capabilities in exploring complex execution paths and generating semantically valid inputs.

A closer examination reveals that the performance gap becomes more pronounced in projects with higher cyclomatic complexity, such as Collections, Compress, and JDatabind. In these projects, test generation requires constructing intricate object states and satisfying implicit preconditions across multiple method calls. Smaller models often fail to capture such dependencies, resulting in significantly lower coverage.

In contrast, larger models are better at reasoning about multi-step object initialization and cross-method interactions, enabling them to trigger hard-to-reach branches. This suggests that while CogPATH effectively structures the test generation process, its full potential is realized when paired with high-capacity LLMs capable of deeper semantic reasoning.

Answer to RQ3

CogPATH generalizes well across different LLMs, consistently achieving strong coverage. However, model capacity plays a critical role in determining effectiveness. Larger models outperform smaller ones by a significant margin (up to **39.1%** in branch coverage), particularly on complex projects requiring multi-step reasoning and implicit state construction.

5 DISCUSSION

Cost and Token Efficiency. CogPATH introduces additional prompting overhead due to backward slicing and constraint hint construction. Across all 3,576 requests, the total token consumption amounts to **221.8 million input tokens** and **6.1 million output tokens**. However, a substantial portion of the input tokens (**181.5 million**)

are served from cache due to repeated prompt structures, leaving only **40.3 million tokens** billed.

Using GPT-5.4-mini pricing, the total cost of our experiments is approximately **\$58**. For a typical project with around 130 focal methods, this corresponds to a cost of approximately **\$2 per project**, demonstrating that CogPATH is cost-effective for practical deployment.

The backward slicing step further improves efficiency by reducing prompt length by approximately **25%**, ensuring that tokens are primarily allocated to semantically relevant context. Together, slicing and caching significantly improve the coverage-per-token ratio compared to naive full-context prompting.

Difference from HITS's Slicing. Program slicing has long been used to identify the subset of program statements relevant to a particular computation [19]. Recent work such as NS-Slicer [14] demonstrates that pre-trained language models can approximate static slices by learning variable-statement dependencies. In the context of LLM-based test generation, HITS [18] applies slicing to guide test synthesis; however, its decomposition is driven by structural heuristics rather than coverage feedback, meaning the resulting segments are not targeted toward any specific uncovered branch. Furthermore, for methods with high cyclomatic complexity, the number of generated segments can grow substantially, incurring significant token overhead regardless of whether the corresponding code has already been covered. In contrast, CogPATH performs *path-aware* backward slicing triggered only when coverage improvement stalls, ensuring that each slice is both necessary and focused on the precise dependencies of the uncovered branch.

Qualitative Insight. To understand the source of performance gains, we examine a representative example from the *Favicon for Alibaba* project. The method `generateFavicon` contains a branch condition `if (icon != null && icon.size() > 0)`, which requires proper object initialization prior to invocation. Baseline approaches typically invoke the focal method directly, failing to satisfy the implicit precondition and leaving the branch uncovered.

In contrast, COGPATH applies backward slicing to identify the relevant state-modifying call `setIconState`, and provides an explicit constraint hint describing the required precondition. The generated test consequently initializes the object correctly and triggers the target branch. This example highlights that COGPATH improves not only coverage, but also the LLM's ability to reason about cross-method dependencies and implicit state requirements.

Design Implications. Our findings suggest that structural guidance (e.g., slicing and constraint hints) and model reasoning capabilities are complementary. While COGPATH effectively reduces irrelevant context and exposes execution constraints, its full potential is realized when paired with LLMs capable of multi-step reasoning and semantic inference. This indicates that future improvements in LLM reasoning ability can further amplify the effectiveness of structured test generation frameworks.

Limitations. Despite its effectiveness, COGPATH has several limitations. *Slice granularity.* For projects such as *Collections* and *Compress*, where branch conditions depend primarily on value ranges rather than cross-method state, backward slicing may remove useful auxiliary information (e.g., boundary constraints), potentially reducing test quality. Conversely, overly coarse slices may introduce irrelevant context. Designing adaptive slicing strategies remains an important direction for future work.

Scalability. Backward slicing and constraint extraction introduce preprocessing overhead proportional to the inter-procedural dependency graph. In our evaluation, the average preprocessing time per focal method is 1.2 s, which is acceptable for medium-scale projects. However, further optimizations such as incremental or on-demand slicing may be required for large-scale industrial systems.

6 Related Work

6.1 Automated Unit Test Generation

Automated unit test generation has been studied through two major paradigms: search-based software testing (SBST) and, more recently, LLM-based synthesis. Early techniques such as Randoop [12] and EvoSuite [5] drive test generation by optimizing structural coverage objectives. EvoSuite applies evolutionary algorithms over whole test suites to maximize branch coverage and automatically infers regression assertions. While effective at exploring input spaces, SBST methods fundamentally operate on syntactic signals and struggle with semantic preconditions that require understanding object states or inter-procedural call sequences [11].

Before the rise of large language models, neural sequence-to-sequence models offered a first bridge between code understanding and test synthesis. ATLAS [?] frames test generation as a translation task, training an encoder-decoder network to predict assert statements from focal methods. AthenaTest [?] extends this direction by fine-tuning pre-trained models on large method-test corpora to automate full JUnit test synthesis. These deep learning approaches demonstrate that neural models can capture surface-level code patterns, but their capacity is fundamentally bounded by training distribution: they cannot reliably generalize to the combinatorial space of runtime behaviors or reason about execution semantics.

The advent of large language models overcomes this boundary by enabling open-ended test synthesis grounded in broad code understanding. ChatUnitTest [3] introduces a focal-context mechanism that selectively exposes program information to the model and adopts a generate-validate-repair loop to improve compilation rates. These early LLM-based approaches demonstrate the potential of neural generation but expose a new challenge: without structured guidance, models tend to produce superficially plausible tests that fail to reach complex branches.

A first wave of follow-up work addresses this gap by incorporating execution signals into the generation loop. CoverUp [13] iteratively targets untested code regions using dynamic coverage reports, directing the model's attention toward uncovered lines, and YATE [?] further repairs failing tests to improve validity. These methods raise coverage by closing the generation-execution loop, but still rely on what the model can observe locally in the method body, leaving cross-method dependencies implicit.

A second wave enriches the model's input with structural program information. SymPrompt [14] extracts branch conditions from Control Flow Graphs (CFGs) and embeds them as structured prompts, helping the model reason about execution paths. HITS [18] decomposes complex methods into smaller slices to reduce cognitive load, improving syntactic validity. Panta [7] combines static CFG analysis with dynamic coverage feedback to identify and target uncovered paths. JUnitGenie [9] further achieves strict path sensitivity by constructing minimal, path-specific contexts using type and data-dependency information. These advances substantially reduce the structural mismatch between model input and coverage objective, yet a fundamental gap persists on branches whose reachability depends on cross-method object states—preconditions established by prior setter or initializer calls that are invisible within the focal method body. Existing approaches conflate constraint inference with test synthesis in a single generation step, leaving the model to silently skip complex dependency chains. Furthermore, even when path information is provided, the full method body dilutes the model's attention with statements irrelevant to the target branch, amplifying context noise as complexity grows [4, 17]. COGPATH directly addresses both dimensions: Constraint-Hints makes implicit cross-method preconditions explicit before generation begins, while coverage-driven Backward Slicing restructures the prompt around branch-relevant dependencies.

6.2 Program Slicing

Program slicing [16, 19] extracts the subset of statements that influence a particular variable or program point. Classical static slicers identify data and control dependencies via program dependence graphs, while dynamic variants such as JavaSlicer [6] track runtime dependencies to produce execution-specific slices. These techniques have been widely applied in debugging, program comprehension, and regression testing. More recently, NSSlicer [20] demonstrates that neural models can approximate static slices even for incomplete programs by learning variable-statement dependency patterns, extending applicability to contexts where traditional analysis is brittle.

In the test generation domain, HITS [18] applies a form of method decomposition to reduce LLM input complexity. However, its decomposition is driven by structural heuristics rather than data and control dependencies, and is not targeted toward any specific uncovered branch; for high-complexity methods the number of generated segments can grow substantially, incurring token overhead regardless of coverage impact. In contrast, COGPATH adopts a stricter, coverage-driven interpretation of backward slicing: starting from an uncovered branch condition, the slicer traces all data and control dependencies—including cross-method field initializations and setter definitions—to produce a focused, branch-specific context. Slicing is triggered selectively only when coverage improvement stalls, ensuring that each slice is both necessary and precisely targeted, rather than a structurally-motivated but coverage-agnostic decomposition.

7 CONCLUSION

In this work, we presented COGPATH, a novel framework for automated unit test generation that leverages Backward Slicing (BS) and Constraint Hints (CS) to address the challenges of context-heavy methods and cross-method preconditions. Through an extensive empirical evaluation on a diverse set of real-world Java projects, we demonstrated that both components contribute independently and complementarily to branch coverage improvement.

Our ablation study confirmed that BS is particularly effective in eliminating irrelevant context, which can mislead LLM-based test generation in complex methods, while CS explicitly guides the LLM to satisfy necessary preconditions that span method boundaries. The combination of BS and CS consistently achieves the highest coverage growth, outperforming variants with one or both components removed, and illustrating that neither component is redundant.

Beyond the immediate improvements in branch coverage, our analysis highlights practical insights for future research. In particular, slice granularity and context selection require careful tuning, and there are opportunities to further enhance constraint-guided LLM reasoning for complex initialization sequences.

Overall, COGPATH provides a promising direction for bridging symbolic analysis techniques with LLM-driven test generation, improving both the effectiveness and reliability of automated testing in real-world software projects. Future work includes extending the approach to handle even more complex object-oriented patterns and integrating additional dynamic program analysis to further refine constraint guidance.

References

- [1] Roberto Baldoni, Emilio Coppa, Daniele Cono D'Elia, Camil Demetrescu, and Irene Finocchi. 2018. A Survey of Symbolic Execution Techniques. arXiv:1610.00502 [cs.SE] <https://arxiv.org/abs/1610.00502>
- [2] Cristian Cadar, Daniel Dunbar, Dawson R Engler, et al. 2008. Klee: unassisted and automatic generation of high-coverage tests for complex systems programs. In *OSDI*, Vol. 8. 209–224.
- [3] Yinghao Chen, Zehao Hu, Chen Zhi, Junxiao Han, Shuiguang Deng, and Jianwei Yin. 2024. *ChatUniTest: A Framework for LLM-Based Test Generation*. arXiv:2305.04764 <http://arxiv.org/abs/2305.04764>
- [4] Yufeng Du, Minyang Tian, Srikanth Ronanki, Subendhu Rongali, Sravan Boda-pati, Aram Galstyan, Azton Wells, Roy Schwartz, Eliu A Huerta, and Hao Peng. 2025. Context Length Alone Hurts LLM Performance Despite Perfect Retrieval. arXiv:2510.05381 [cs.CL] <https://arxiv.org/abs/2510.05381>

- [5] Gordon Fraser and Andrea Arcuri. 2011. EvoSuite: automatic test suite generation for object-oriented software. In *Proceedings of the 19th ACM SIGSOFT Symposium and the 13th European Conference on Foundations of Software Engineering (Szeged, Hungary) (ESEC/FSE '11)*. Association for Computing Machinery, New York, NY, USA, 416–419. doi:10.1145/2025113.2025179
- [6] Carlos Galindo, Sergio Perez, and Josep Silva. 2022. A Program Slicer for Java (Tool Paper). In *Software Engineering and Formal Methods*, Bernd-Holger Schlingloff and Ming Chai (Eds.). Springer International Publishing, Cham, 146–151.
- [7] Sijia Gu, Noor Nashid, and Ali Mesbah. 2025. LLM Test Generation via Iterative Hybrid Program Analysis. doi:10.48550/arXiv.2503.13580 arXiv:2503.13580
- [8] René Just, Darioush Jalali, and Michael D. Ernst. 2014. Defects4J: a database of existing faults to enable controlled testing studies for Java programs. In *Proceedings of the 2014 International Symposium on Software Testing and Analysis (San Jose, CA, USA) (ISSTA 2014)*. Association for Computing Machinery, New York, NY, USA, 437–440. doi:10.1145/2610384.2628055
- [9] Dianshu Liao, Xin Yin, Shidong Pan, Chao Ni, Zhenchang Xing, and Xiaoyu Sun. 2025. Navigating the Labyrinth: Path-Sensitive Unit Test Generation with Large Language Models. doi:10.48550/arXiv.2509.23812 arXiv:2509.23812
- [10] T.J. McCabe. 1976. A Complexity Measure. *IEEE Transactions on Software Engineering* SE-2, 4 (1976), 308–320. doi:10.1109/TSE.1976.233837
- [11] Phil McMinn. 2011. Search-Based Software Testing: Past, Present and Future. In *2011 IEEE Fourth International Conference on Software Testing, Verification and Validation Workshops*. 153–163. doi:10.1109/ICSTW.2011.100
- [12] Carlos Pacheco and Michael D. Ernst. 2007. Randoop: feedback-directed random testing for Java. In *Companion to the 22nd ACM SIGPLAN Conference on Object-Oriented Programming Systems and Applications Companion (Montreal, Quebec, Canada) (OOPSLA '07)*. Association for Computing Machinery, New York, NY, USA, 815–816. doi:10.1145/1297846.1297902
- [13] Juan Altmayer Pizzorno and Emery D. Berger. 2024. *CoverUp: Coverage-Guided LLM-Based Test Generation*. arXiv:2403.16218 <http://arxiv.org/abs/2403.16218>
- [14] Gabriel Ryan, Siddhartha Jain, Mingyue Shang, Shiqi Wang, Xiaofei Ma, Murali Krishna Ramanathan, and Baishakhi Ray. 2024. Code-Aware Prompting: A Study of Coverage-Guided Test Generation in Regression Setting Using LLM. *Proceedings of the ACM on Software Engineering* 1, FSE (July 2024), 951–971. doi:10.1145/3643769
- [15] The Apache Software Foundation. 2016. Apache Commons Math 3.6.1. <https://commons.apache.org/proper/commons-math/>. Version 3.6.1.
- [16] Frank Tip. 1995. A survey of program slicing techniques. *J. Program. Lang.* 3, 3 (1995). <http://compsinet.dcs.kcl.ac.uk/JP/jp030301.abs.html>
- [17] Weiwei Wang, Jiyong Min, and Weijie Zou. 2026. Intelligence Degradation in Long-Context LLMs: Critical Threshold Determination via Natural Length Distribution Analysis. arXiv:2601.15300 [cs.CL] <https://arxiv.org/abs/2601.15300>
- [18] Zejun Wang, Kaibo Liu, Ge Li, and Zhi Jin. 2024. HITS: High-coverage LLM-based Unit Test Generation via Method Slicing. In *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering (Sacramento CA USA, 2024-10-27)*. ACM, 1258–1268. doi:10.1145/3691620.3695501
- [19] Mark Weiser. 1984. Program Slicing. *IEEE Trans. Softw. Eng.* 10, 4 (July 1984), 352–357. doi:10.1109/TSE.1984.5010248
- [20] Aashish Yadavally, Yi Li, Shaohua Wang, and Tien N. Nguyen. 2024. A Learning-Based Approach to Static Program Slicing. *Proceedings of the ACM on Programming Languages* 8, OOPSLA1 (April 2024), 83–109. doi:10.1145/3649814
- [21] Chen Yang, Junjie Chen, Bin Lin, Jianyi Zhou, and Ziqi Wang. 2024. Enhancing LLM-based Test Generation for Hard-to-Cover Branches via Program Analysis. *arXiv preprint arXiv:2404.04966* (2024).

Received 20 February 2007; revised 12 March 2009; accepted 5 June 2009